

I2C implementation using FT232R

Here is the datasheet of ft232r [Future Technology Devices International Ltd. FT232R USB UART IC Datasheet](#)

Task 1: Write Python code to read and write Data from I2C based EEPROM through FT232R. Capture the LA waveform

Takeaways from datasheet :-

Voltage on Any Pin with Respect to Ground-1.0V to +7.0V Maximum
Operating Voltage 6.25V

The write protect input, when connected to GND, allows normal write operations
When WP is connected high to VCC, all write operations to the memory are inhibited

Data on the SDA pin may change only during SCL low time periods

The eighth bit of the device address is the read/write operation select bit. A read operation is initiated if this bit is high(1) and a write operation is initiated if this bit is low (0).

Task 1 experiments i have done : -

This is the wiring that i done in breadboard

AT24C256 (notch up)

Pin 1 A0 -> GND

Pin 2 A1 -> GND

Pin 3 A2 -> GND

Pin 4 GND -> FT232R GND

Pin 5 SDA -> FT232R TX

-> FT232R RX

-> LA CH0

Pin 6 SCL -> FT232R DTR

-> LA CH1

Pin 7 WP -> GND

Pin 8 VCC -> FT232R VCC

Pull-ups: 4.7k resistor: VCC -> SDA, 4.7k resistor:VCC -> SCL

Logic Analyzer:

CH0 -> SDA

CH1 -> SCL

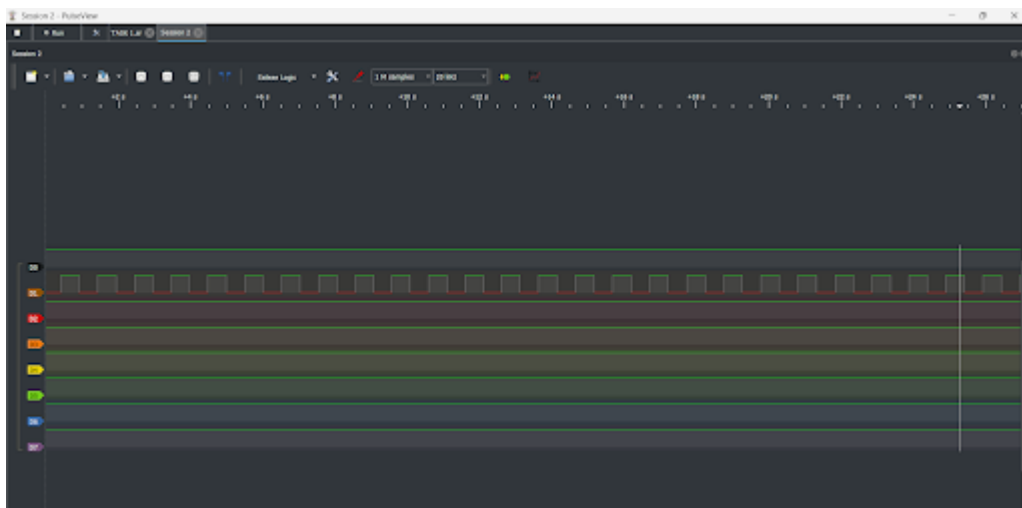
GND -> FT232R GND

STEP 1 : dtr loop check test

Code :

```
import serial
import time
ser = serial.Serial("COM9")
while True:
    ser.setDTR(True)
    time.sleep(0.5)
    ser.setDTR(False)
    time.sleep(0.5)
```

Got this in **Pulse View Output :**



D1 (SCL) is pulsing cleanly: In the line 2 we got square wave alternating high and low every 0.5 seconds. This shows that python has control over DTR pin from FT232R

D0 (SDA) is sitting rock-solid HIGH: In the first line we can see flat high line, 4.7k ohms pull up resistor is working and the voltage from vcc is not pulled down

Conclusion :

EEPROM is powered, Pull-up resistors are working, SDA is being pulled HIGH ,
DTR is successfully toggling SCL

`ser.setDTR(True)` pulls the SCL line LOW, and `ser.setDTR(False)` releases it HIGH.

STEP 2 : 55 to binary check test

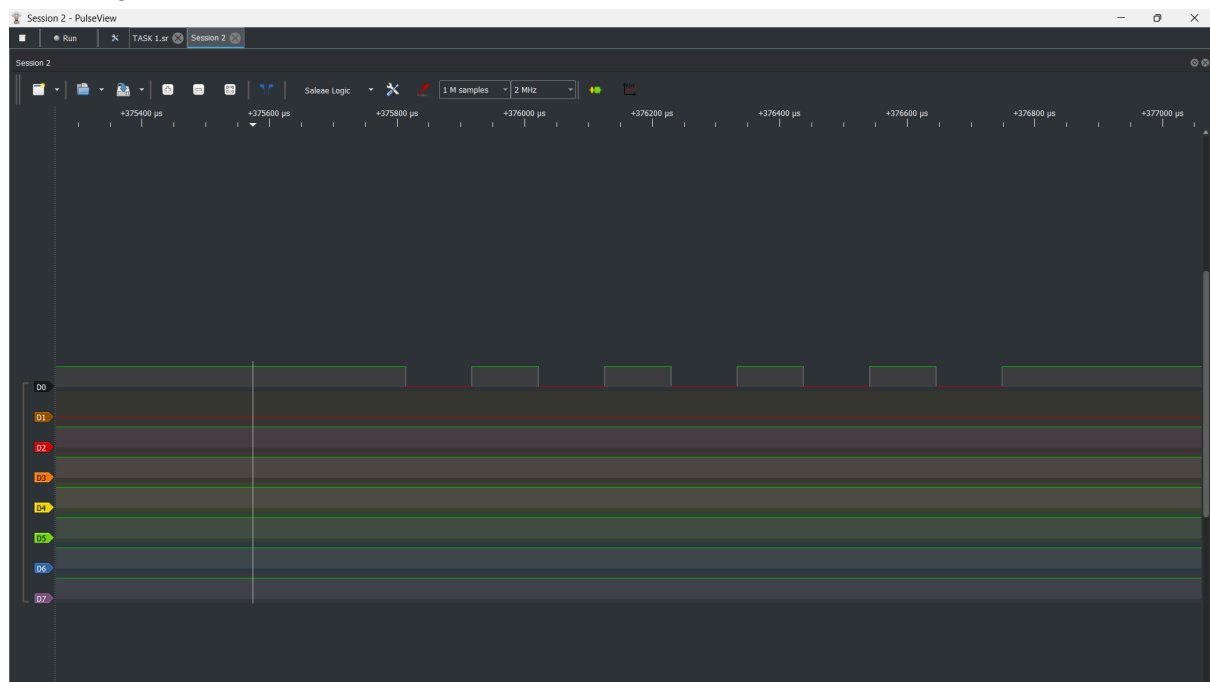
Code :

```
import serial
import time
```

```
ser = serial.Serial("COM9", 9600)
```

```
while True:
    ser.write(b'\x55')
    time.sleep(1)
```

Got this graph



D0 is pulsing up and down cleanly.

In the code we sended the \x55 which is 01010101 ,so the TX can move from 5v to 0v
Both the pull ups and chip are strong to change the voltage from high to low

In the D1 it is only low red line it is connected to dtr and the code doesnt said to change the dtr so it is low

Conclusion : TX line can be driven LOW, TX line can be driven HIGH,

Logic Analyzer sees it, Pull-up resistor isn't preventing TX from pulling the line low

STEP 3 : python break condition test

code

```
import serial
import time
```

```
ser = serial.Serial("COM9")
```

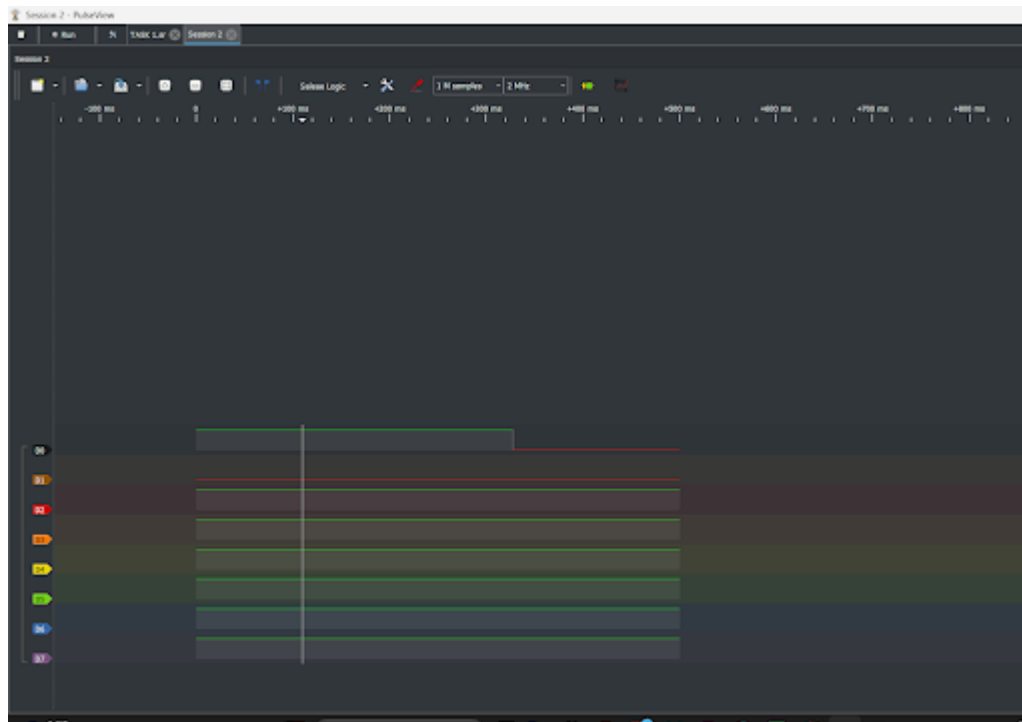
```

while True:
    ser.break_condition = True
    print("LOW")
    time.sleep(1)

    ser.break_condition = False
    print("HIGH")
    time.sleep(1)

```

Got this



In the D0 we are making it from high to low
 So the D1 is for DTR which is not changing so it is low

- `break_condition = True` actively forces the SDA wire to **LOW**, and
`break_condition = False` places the wire in a high-impedance state to let the pull-up snap it back to **HIGH**.

STEP 4 : Start condition check test

Code :

```

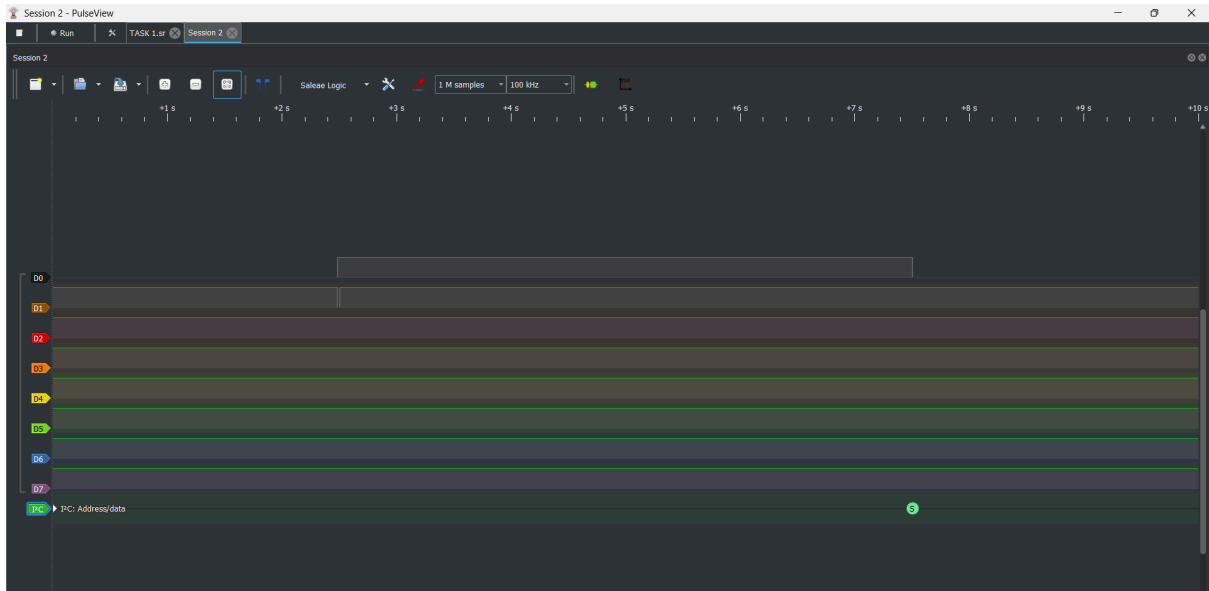
import serial
import time
ser = serial.Serial("COM9")
print("Idle")
ser.setDTR(False) # SCL High
ser.break_condition = False # SDA High

```

```

time.sleep(5)
print("START")
ser.break_condition = True # SDA falls LOW while SCL stays High
time.sleep(5)
print("Done")
input("Press Enter...")

```



In the D0 we got low to high at 2.5 s and to low at 7.5 s this is the break condition
 In case of D1 since we set the dtr as false it become high and so on
 At the end we can see "s"

STEP 5 : single byte test write

Code :

```

import serial
import time

```

```

ser = serial.Serial("COM9")

```

DTR inverted on your setup

```

def scl_high():
    ser.setDTR(False)

```

```

def scl_low():
    ser.setDTR(True)

```

break_condition=True -> SDA LOW

break_condition=False -> SDA HIGH

```

def sda_high():
    ser.break_condition = False

```

```

def sda_low():

```

```

    ser.break_condition = True

DELAY = 0.02

def i2c_start():
    sda_high()
    scl_high()
    time.sleep(DELAY)

    sda_low()
    time.sleep(DELAY)

    scl_low()
    time.sleep(DELAY)

def i2c_stop():
    sda_low()
    scl_high()
    time.sleep(DELAY)

    sda_high()
    time.sleep(DELAY)

def send_bit(bit):
    if bit:
        sda_high()
    else:
        sda_low()

    time.sleep(DELAY)

    scl_high()
    time.sleep(DELAY)

    scl_low()
    time.sleep(DELAY)

def send_byte(val):
    for i in range(7, -1, -1):
        send_bit((val >> i) & 1)

# -----
# EEPROM WRITE DEMO
# -----

# AT24C256 address with A0=A1=A2=0
DEVICE_WRITE = 0xA0

```

```

i2c_start()

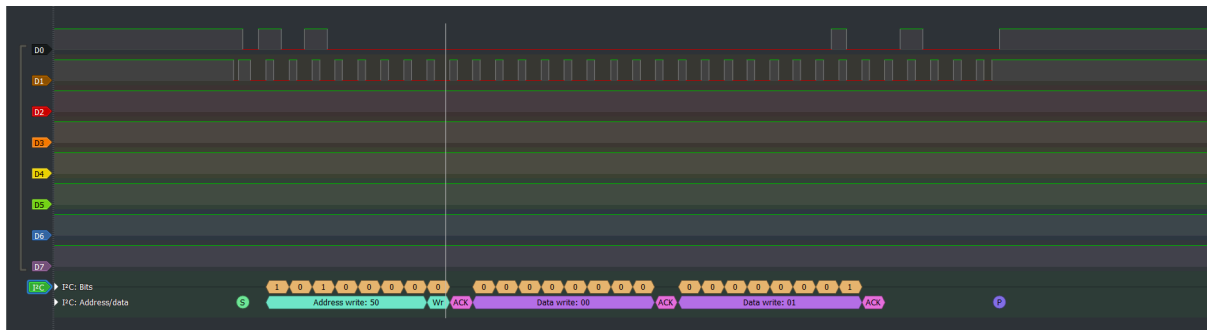
send_byte(DEVICE_WRITE) # EEPROM address
send_byte(0x00)         # memory address high
send_byte(0x00)         # memory address low
send_byte(ord('H'))     # write H

i2c_stop()

print("Done")
input("Press Enter...")

```

Got this



Here as i mentioned before the ch0 is sda and ch1 is scl
The start condition occurs when scl is high and sda is gone from high to low

0x50 is the 7-bit I2C address of the EEPROM. In the I2C address byte, this 7-bit value is shifted left by one bit and the last bit is used for read/write selection. So $0x50 \ll 1 = 0xA0$ for write

Then ack came from the eeprom
The first address byte ended with bit 0, so it was a write
Then 8 pulses came the last bit came as 1
At the stop condition the pulse become high as usual

STEP 6 : full read write test

Code :

```

import serial
import time

ser = serial.Serial("COM9")

def scl_high(): ser.setDTR(False)
def scl_low(): ser.setDTR(True)
def sda_high(): ser.break_condition = False

```

```

def sda_low(): ser.break_condition = True

DELAY = 0.02

def i2c_start():
    sda_high(); scl_high(); time.sleep(DELAY)
    sda_low(); time.sleep(DELAY)
    scl_low(); time.sleep(DELAY)

def i2c_stop():
    sda_low(); scl_high(); time.sleep(DELAY)
    sda_high(); time.sleep(DELAY)

def send_bit(bit):
    if bit: sda_high()
    else: sda_low()
    time.sleep(DELAY); scl_high(); time.sleep(DELAY); scl_low(); time.sleep(DELAY)

def send_byte(val):
    for i in range(7, -1, -1):
        send_bit((val >> i) & 1)
    # Master releases SDA to allow slave ACK
    sda_high()
    time.sleep(DELAY); scl_high(); time.sleep(DELAY); scl_low(); time.sleep(DELAY)

# -----
# COMPREHENSIVE WRITE-THEN-READ TRANSACTION
# -----
# Step 1: Write Byte 'H' (0x48) to memory index 0x0000
i2c_start()
send_byte(0xA0) # Device Address + Write
send_byte(0x00) # Mem Address High
send_byte(0x00) # Mem Address Low
send_byte(0x48) # Data Payload ('H')
i2c_stop()

print("Write complete. Waiting for internal EEPROM write cycle...")
time.sleep(0.05) # Safe delay for EEPROM internal flash writing

# Step 2: Random Read back from memory index 0x0000
print("Executing Read Back protocol...")
i2c_start()
send_byte(0xA0) # Dummy Write to set the internal pointer to 0x0000
send_byte(0x00)
send_byte(0x00)

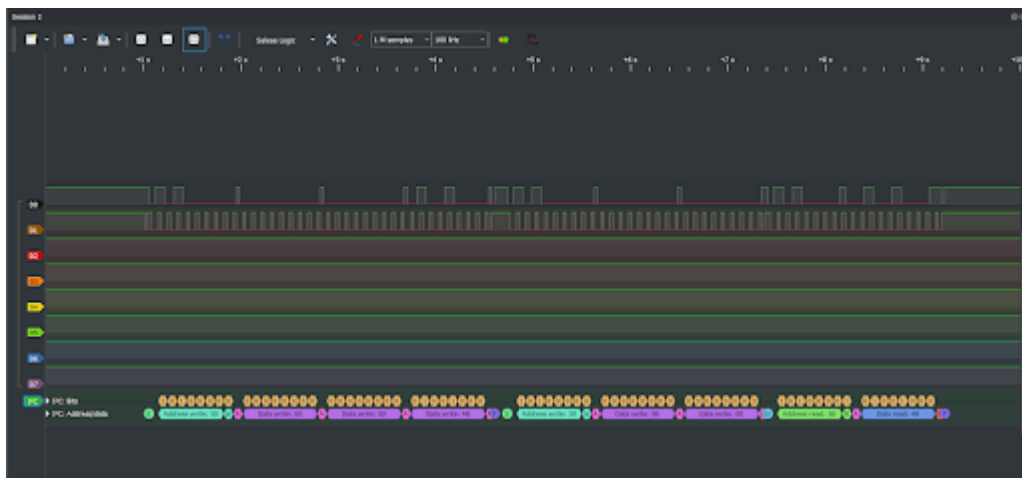
i2c_start() # Repeated START Condition
send_byte(0xA1) # Device Address + READ Token (0x50 shifted with Read bit 1)

```

```
# Generate 8 clock pulses to let the EEPROM drive the SDA line back
for i in range(8):
    time.sleep(DELAY); scl_high(); time.sleep(DELAY); scl_low(); time.sleep(DELAY)

# Master sends NACK (SDA High) to terminate read session
sda_high(); time.sleep(DELAY); scl_high(); time.sleep(DELAY); scl_low();
time.sleep(DELAY)
i2c_stop()

print("Done. Check PulseView for the 0xA1 Read block!")
```



Here as usual start condition came then last bit is zero so it write
 After each 8 bit acknowledgement came from the slave
 First 2 8 bits are data write 00 00 it is the memory index location then the data write
 becomes 48 which is letter H then stop condition
 In the read sequence same data write 00 00 happens and sr represents the repeated start
 without acknowledgement then ee prom reads the data